RabbitMQ Messaging: Review & Changes Report

auth-modular (Laravel modular monolith) · branch `main`, uncommitted · 25 October 2026

7	35	+1,175 / −276	40 + E2E
pre-existing bugs fixed	files changed	lines added / removed	checks passing, incl. real broker

This report covers the review of the RabbitMQ messaging layer and every change made in response: what changed and why, what was added or removed, and how the structure looks now. Besides the code, one change was made on the RabbitMQ broker itself (section 5).

Overview

The work happened in three rounds:

Round	Goal	Review items
1	Stop losing messages and crashing the worker	Fixes 1–5, plus 2 bugs found while testing
2	Make publishing reliable	6 (publisher confirms), 7 (transactional outbox)
3	Clean module structure and production-ready workers	8–9 (module subscriptions), 10 (config), 11 (heartbeats, reconnect, shutdown)

Bugs fixed at a glance

Bug	Impact	Fix
Retries were sent back to `modular.events` with routing key `retry`, which nothing binds	Every retry was silently dropped	Retry queue now dead-letters straight back to its own queue
Duplicate check keyed on `message_id` only	A second module subscribed to the same event would skip it	Keyed on `(queue, message_id)`
Broker nack on the retry publish was ignored	Original message acked and lost	Nack handler turns it into a failure, so the original is requeued
JSON decoding happened outside the `try`	One malformed message crashed the worker in an endless loop	Decode inside the `try`; malformed messages go to the DLQ
No prefetch limit	Whole queue pushed into one worker's memory; no load sharing	`basic_qos` prefetch of 10
`nack(false, true)`: arguments in the wrong order	Message dead-lettered, plus every other unacked message on the channel rejected	Named arguments: `nack(requeue: true)`
`Message::$messageId` was a promoted readonly property reassigned in the constructor	Every `new Message(...)` threw on PHP 8.3/8.4, so nothing could be published	Declared as a normal readonly property

1. Changed files

Shared: RabbitMQ infrastructure

RabbitMQTopology.php

Change	Why
The retry queue now sends expired messages to the default exchange, addressed to the original queue	Retries were silently dropped. They went back to `modular.events` with the routing key `retry`, and nothing binds that key. This also stops a retry from being re-delivered to every subscribing module.
Exchange name and retry delay come from config	They were hardcoded.
`RabbitMQConnection` is passed into the constructor	It used the `app()` service locator, unlike the other classes.

RabbitMQConsumer.php (rewritten)

Change	Why
JSON is decoded inside the `try`, and malformed messages go straight to the DLQ	One bad message crashed the worker. It was redelivered and crashed it again, in an endless loop.
`basic_qos` prefetch limit (10)	RabbitMQ was pushing the whole queue into one worker's memory, and extra workers couldn't share the load.
Duplicate check uses `(queue, message_id)`	With the check on `message_id` alone, a second module subscribed to the same event would think it was already handled and skip it.
`nack(requeue: true/false)` named arguments	`nack(false, true)` actually meant "don't requeue, and reject every unacked message on the channel".
A broker rejection of the retry publish is now treated as a failure	php-amqplib ignores broker nacks unless you register a handler, so the original was acked and the message lost.
`ack()` moved outside the handler's `try`	A dropped connection during ack was being mistaken for a handler failure.
Reconnect loop (1s → 30s backoff), `stop()`, limits, and an idle check every 5s	The worker died on any connection drop, couldn't shut down cleanly, and had no restart limits.
Retry count comes from config	It was hardcoded as `MAX_RETRIES = 3`.

RabbitMQPublisher.php

Change	Why
Waits for broker confirmation, with the mandatory flag set	It used to send and forget: `publish()` reported success even when the broker dropped the message.
Connection is opened once and reused	Every publish opened a new TLS connection: about 2.5s on CloudAMQP, versus about 0.4s reused.
Checks the heartbeat before use, and retries once on a fresh connection	A connection left idle and closed by the broker would otherwise make that publish fail.
Disconnects after any failure	The old code leaked the connection when an exception was thrown.
Logs "no subscribers" instead of failing	An event nobody listens to is normal, and failing would block the outbox.

RabbitMQConnection.php

Change	Why
Connects through `AMQPConnectionConfig` and the factory	`AMQPSSLConnection` is deprecated and will be removed in php-amqplib v4.
60s heartbeat, with socket timeouts derived from it	There was no heartbeat, so dead connections weren't detected.
Connection name set to the app name	Makes the connection identifiable in the RabbitMQ management UI.

Shared: application layer and wiring

File	Change	Why
`Message.php`	`messageId` is no longer a promoted readonly property	Every `new Message(...)` threw on PHP 8.3 and 8.4, so nothing could be published.
`MessageConsumerInterface.php`	`consume()` takes `ConsumerLimits`; new `stop()` method	Needed for clean shutdown and restart limits.
`MessagePublisherInterface.php`	Documentation only	Explains that publishing now goes through the outbox, so call it inside your DB transaction.
`SharedServiceProvider.php`	Interface → `OutboxPublisher`; `RabbitMQPublisher` is given only to `OutboxRelay`; `MessageSubscriptions` singleton	Application code publishes into the outbox, and only the relay talks to the broker.

Modules

File	Change	Why
`UserRegistered.php`	Moved from `Auth/Domain/Events` to `Auth/IntegrationEvents`, and gained `fromPayload()`	It's Auth's public contract. Other modules may import it, but never Auth's Domain.
`AuthEventPublisher.php`	Import path only	Follows the move.
`UserRegisteredHandler.php`	Implements the shared `MessageHandlerInterface` and uses a typed event	Replaces raw `$message->payload['email']` access. Your `message_id` log line is kept.
`NotificationsServiceProvider.php`	Declares its queue and event → handler map	The module now owns its subscriptions. They used to live in a global command.

Commands, config and routes

File	Change	Why
`SetupRabbitMQTopology.php`	Builds the topology from module subscriptions and prints it	The queue name and bindings were hardcoded.
`PublishRabbitMQMessage.php` `PublishUserRegisteredTest.php`	Output text, plus an import path	They now say "stored in the outbox, run the relay" instead of "published".
`config/rabbitmq.php`	New keys (listed in section 5)	Values that were hardcoded can now be configured.
`routes/console.php`	Daily `model:prune` for the outbox	Stops the outbox table growing forever.

2. New files

File	What it is	Why
`PublishesWithConfirms.php`	Trait with the message format and confirmation handling	The publisher and the consumer's retry share one copy of this tricky code.
`Outbox/OutboxMessage.php`	Eloquent model for `outbox_messages`, prunable after 7 days	Stores events until the relay ships them.
`Outbox/OutboxPublisher.php`	Default `MessagePublisherInterface`; writes a row	Your data change and its event commit together or not at all.
`Outbox/OutboxRelay.php`	Sends pending rows in order and stops at the first failure	Delivers events after commit, and later events can't overtake earlier ones.

File	What it is	Why
`MessageSubscriptions.php`	Registry of queue → {event → handler}	Each subscription is declared once; queue bindings are derived from it.
`MessageDispatcher.php`	Looks up and runs the subscribed handler	Replaces the hand-written Notifications dispatcher and logs unknown events.
`MessageHandlerInterface.php`	Shared handler contract	Every module's handlers have the same shape.
`ConsumerLimits.php`	Max messages, seconds and memory	Lets Supervisor recycle workers.
`ConsumeMessages.php`	`rabbitmq:consume {queue}`	One generic worker for any module's queue, with clean shutdown.
`RelayOutboxMessages.php`	`rabbitmq:relay-outbox`	Runs the relay as a long-lived process.
`2026_10_25_120000_add_queue_to_processed_messages_table.php`	Migration	Duplicate check per queue (fix 2).
`2026_10_25_130000_create_outbox_messages_table.php`	Migration	Outbox storage.

3. Removed files

File	Why
`Notifications/…/NotificationEventConsumer.php`	Replaced by `rabbitmq:consume` and `MessageDispatcher`.
`Notifications/…/NotificationEventDispatcher.php`	Replaced by `MessageDispatcher`. Its handler map moved into the provider.
`Notifications/…/NotificationEventHandler.php`	Replaced by the shared `MessageHandlerInterface`.
`Console/Commands/ConsumeNotificationEvents.php`	Replaced by `rabbitmq:consume notifications.events.queue`.

4. Structure changes

Yes, the structure changed. In short:

- **New folder** `Shared/Infrastructure/Messaging/Outbox/` holds the transactional outbox (model, publisher, relay). Publishing now goes through the outbox table, and only the relay talks to the broker.
- **New folder** `Auth/IntegrationEvents/` is Auth's public contract. `UserRegistered` moved here from `Auth/Domain/Events`.
- **Notifications lost its own messaging classes.** Its consumer, dispatcher and handler interface are gone; it declares its subscriptions in `NotificationsServiceProvider`, served by the shared `MessageSubscriptions` registry and `MessageDispatcher`.
- **One generic worker command**, `rabbitmq:consume {queue}`, replaces per-module consumer commands.

How a message flows now

```
PUBLISH
  DB::transaction → AuthEventPublisher → MessagePublisherInterface (= OutboxPublisher)
                                            └→ outbox_messages row (commits with your data)
  rabbitmq:relay-outbox → OutboxRelay → RabbitMQPublisher ──confirmed──→ modular.events (topic)

CONSUME
  modular.events ──auth.user.registered──→ notifications.events.queue
  rabbitmq:consume notifications.events.queue
      → RabbitMQConsumer → MessageDispatcher → UserRegisteredHandler
  handler throws  → {queue}.retry.exchange → {queue}.retry.queue (5s) → back to {queue}   (up to 3×)
  retries used up, or malformed → {queue}.dlx → {queue}.dlq
```

New structure

Legend: + new, ~ changed, → moved, × removed. Only messaging-related files are shown.

```
app/Console/Commands/
├── ConsumeMessages.php                          + rabbitmq:consume {queue}
├── RelayOutboxMessages.php                      + rabbitmq:relay-outbox
├── SetupRabbitMQTopology.php                    ~ reads module subscriptions
├── PublishRabbitMQMessage.php                   ~ output text
├── PublishUserRegisteredTest.php                ~ import + output text
├── ConsumeRabbitMQTest.php
├── ConsumeRabbitMQFailureTest.php
├── TestRabbitMQConnection.php
└── ConsumeNotificationEvents.php                × replaced by rabbitmq:consume

app/modules/Auth/
├── Application/Services/AuthEventPublisher.php   ~ import path
├── Domain/Events/UserRegistered.php              → moved to IntegrationEvents/
└── IntegrationEvents/                            + public contract for other modules
    └── UserRegistered.php                        → moved here, + fromPayload()

app/modules/Notifications/
├── Application/Handlers/
│   ├── UserRegisteredHandler.php                 ~ shared interface, typed event
│   └── NotificationEventHandler.php              ×
├── Application/Services/
│   ├── NotificationEventConsumer.php             ×
│   └── NotificationEventDispatcher.php           ×
└── Providers/NotificationsServiceProvider.php    ~ declares queue + subscriptions
```

```
app/modules/Shared/
├── Application/Messaging/
│   ├── Contracts/
│   │   ├── MessageConsumerInterface.php                        ~ + limits, + stop()
│   │   ├── MessageHandlerInterface.php                         +
│   │   ├── MessagePublisherInterface.php                       ~ docs only
│   │   └── MessageTopologyInterface.php
│   ├── ConsumerLimits.php                                      +
│   ├── Message.php                                             ~ readonly bug fix
│   ├── MessageDispatcher.php                                   +
│   ├── MessageSubscriptions.php                                +
│   └── QueueTopology.php
├── Infrastructure/Messaging/
│   ├── Outbox/                                                 + new folder
│   │   ├── OutboxMessage.php                                   +
│   │   ├── OutboxPublisher.php                                 +
│   │   └── OutboxRelay.php                                     +
│   └── RabbitMQ/
│       ├── PublishesWithConfirms.php                           +
│       ├── RabbitMQConnection.php                              ~
│       ├── RabbitMQConsumer.php                                ~
│       ├── RabbitMQPublisher.php                               ~
│       └── RabbitMQTopology.php                                ~
└── Providers/SharedServiceProvider.php                         ~ new bindings

config/rabbitmq.php                                             ~ new keys
routes/console.php                                              ~ daily outbox prune

database/migrations/
├── 2026_10_25_082245_create_processed_messages_table.php       (yours, untracked)
├── 2026_10_25_120000_add_queue_to_processed_messages_table.php +
└── 2026_10_25_130000_create_outbox_messages_table.php          +
```

Dependency rule between modules

A module may import `Shared\Application\*` and other modules' `IntegrationEvents\*`. It must never import another module's `Domain`, `Application` or `Infrastructure`. Notifications now depends only on `Auth\IntegrationEvents\UserRegistered`, not on Auth's Domain, and no event name is typed out by hand.

Publishing modules expose what they emit in `IntegrationEvents/`; consuming modules declare what they react to in their own service provider:

```
$this->callAfterResolving(MessageSubscriptions::class, function (MessageSubscriptions $subscriptions): void {
    $subscriptions->subscribe('notifications.events.queue', [
        UserRegistered::NAME => UserRegisteredHandler::class,
    ]);
});
```

5. Commands, config, database and broker

Commands

Command	Status
`rabbitmq:consume {queue} [--max-messages] [--max-time] [--memory=128]`	New; replaces `rabbitmq:consume-notifications`
`rabbitmq:relay-outbox [--once] [--batch=100] [--sleep=1]`	New
`rabbitmq:setup`	Now built from module subscriptions
`rabbitmq:publish-test`, `rabbitmq:publish-user-registered`	Now write to the outbox

New config keys

Key	Env variable	Default	Purpose
`heartbeat`	`RABBITMQ_HEARTBEAT`	60	Seconds between heartbeats; 0 disables
`connection_timeout`	`RABBITMQ_CONNECTION_TIMEOUT`	10	Seconds to wait when connecting
`exchange`	`RABBITMQ_EXCHANGE`	`modular.events`	Topic exchange for all integration events
`prefetch_count`	`RABBITMQ_PREFETCH_COUNT`	10	Max unacked deliveries per consumer
`confirm_timeout`	`RABBITMQ_CONFIRM_TIMEOUT`	5	Seconds to wait for a publish confirmation
`retry.max_retries`	`RABBITMQ_RETRY_MAX_RETRIES`	3	Retries before dead-lettering
`retry.delay_ms`	`RABBITMQ_RETRY_DELAY_MS`	5000	Delay between retries (the retry queue's TTL)

> If you change `retry.delay_ms`, delete the `*.retry.queue` queues and run `rabbitmq:setup` again. RabbitMQ refuses to redeclare a queue with different arguments.

Database

- `processed_messages`: new `queue` column. The unique key on `message_id` became a unique key on `(queue, message_id)`.
- `outbox_messages`: new table (`message_id`, `name`, `payload`, `headers`, `correlation_id`, `attempts`, `last_error`, `published_at`).

Broker

- Deleted `notifications.events.queue.retry.queue` (it was empty) and recreated it with the new routing settings via `rabbitmq:setup`.

6. Changes that affect you

- `rabbitmq:consume-notifications` no longer exists. Use `rabbitmq:consume notifications.events.queue`.
- Run **one** `rabbitmq:relay-outbox` process under Supervisor, otherwise events never reach RabbitMQ. Also run the Laravel scheduler, so the outbox gets pruned.
- `MessageConsumerInterface` now requires `stop()`.
- `UserRegistered` moved to `App\Modules\Auth\IntegrationEvents`.
- An idle consumer takes up to 5s to notice SIGTERM, so set Supervisor's `stopwaitsecs` to your slowest handler time plus 5s.

Example Supervisor config

```
[program:notifications-consumer]
command=php /path/to/artisan rabbitmq:consume notifications.events.queue --max-time=3600
autorestart=true
stopwaitsecs=30

[program:outbox-relay]
command=php /path/to/artisan rabbitmq:relay-outbox
numprocs=1
autorestart=true
```

> **Not part of these changes:** the `message_id` log field in `UserRegisteredHandler` is your own edit, which was kept. The `create_processed_messages` migration is also yours, has never been committed, and is needed by the new migration.

7. Testing

- **Harnesses:** 40 checks across three scratch harnesses (registry, consumer, outbox) pass on PHP 8.3 and 8.4.
- **Real worker processes:** clean exit on SIGTERM (about 4s when idle, about 0.1s during reconnect backoff), `--max-time` respected, and an unknown queue name fails with the list of valid queues.
- **Real broker, end to end:** a normal event went outbox → relay → queue → handler. A malformed event was retried 3 times about 5s apart and then landed in the DLQ. Everything the test created was cleaned up.
- **Connection:** 60s heartbeat negotiated with CloudAMQP over verified TLS 1.3; the `AMQPSSLConnection` deprecation warning is gone.
- **Test suite:** `php artisan test` passes.
- **Not tested:** the connection dropping mid-stream on the live broker, and the stale-connection check after more than 2 minutes idle.

Still open

- The `test.queue` test commands use a queue that `rabbitmq:setup` never creates.
- The old `create_processed_messages` migration has a blank line before `<?php`.
- The folder is `app/modules` but the namespace is `App\Modules`; module loading breaks on case-sensitive (Linux) filesystems.
- Turn the scratch harnesses into real Pest tests.